MapReduce

Dean and Ghemawat. MapReduce: Simplified Data Processing on Large Clusters. Communications of the ACM, Vol. 51, No. 1, January 2008.

Shahram Ghandeharizadeh
Computer Science Department
University of Southern California

Primary Functionality of Google

Primary Functionality of Google

- Search content on the web in browsing mode.
- Open world assumption: If your search with Google does not return results, it does *not* mean that the referenced content does not exist. It only means that Google does not know about it when the search was issued.
  - Google may produce results if the search is issued again.
  - Do not index/find me: Google provides tags to enable an information provider to prevent Google from indexing its pages.
  - No one gets angry if Google does not retrieve information known to exist on the Internet.

How is this different than financial applications?

Functionality

IR: Google

- Search content on the web in browsing mode.
- Open world assumption: If your search with Google does not return results, it does *not* mean that the referenced content is non-existent. It only means that Google does not know about it when the search was issued.
  - Google may retrieve results if the search is issued again.
  - Do not index/find me: Google provides tags to enable an information provider to prevent Google from indexing its pages.
  - No one gets angry if Google does not retrieve information known to exist on the Internet.

DB:

- Query based: Looking for a needle in a haystack.
- Closed world assumption: A data item that is not known does not exist.
  - A query must retrieve correct results 100% of the time!
  - If a customer insists the bank cannot find their account because the customer has used Google's "do not find me" tags, the customer is kicked out!

  - Customers become angry if the system retrieves incorrect data.

Key Observation

IR: Google

- Okay to return either no or incorrect results.
- Acceptable for a user search to observe stale data.

DB: 

- Not okay to return incorrect results.
- A transaction must produce consistent results.
- A query interface, e.g., SQL, simple key-value interface, etc.

Key Observation

IR: Google

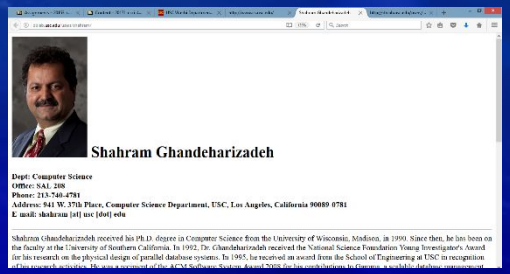
- Okay to return either no or incorrect results.
- Acceptable for a user search to observe stale data.

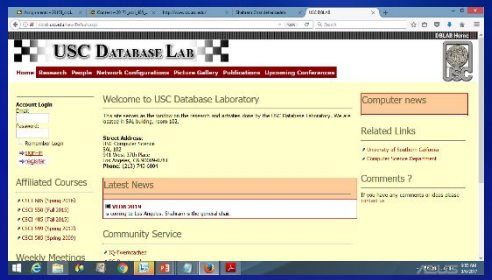
DB: 

- Not okay to return incorrect results.
- A transaction must produce consistent results.
- A query interface, e.g., SQL, simple key-value interface, etc.

How to support Internet search/information retrieval?

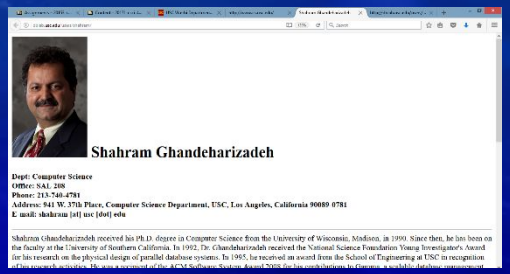
Web Crawlers

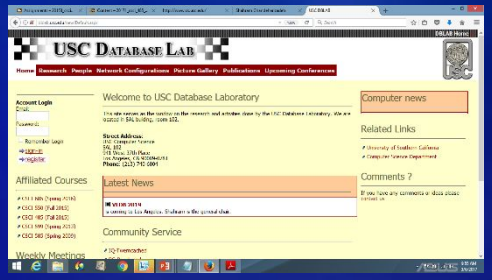




Read HTML pages periodically and index them.

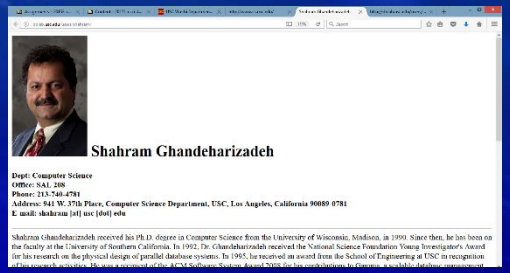
Web Crawlers

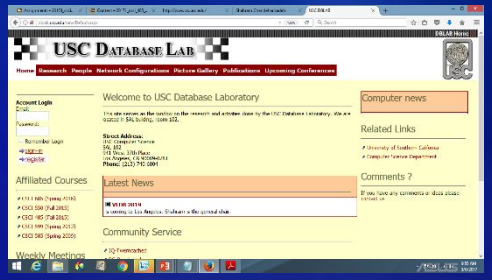




Why not an RDBMS server?

Web Crawlers



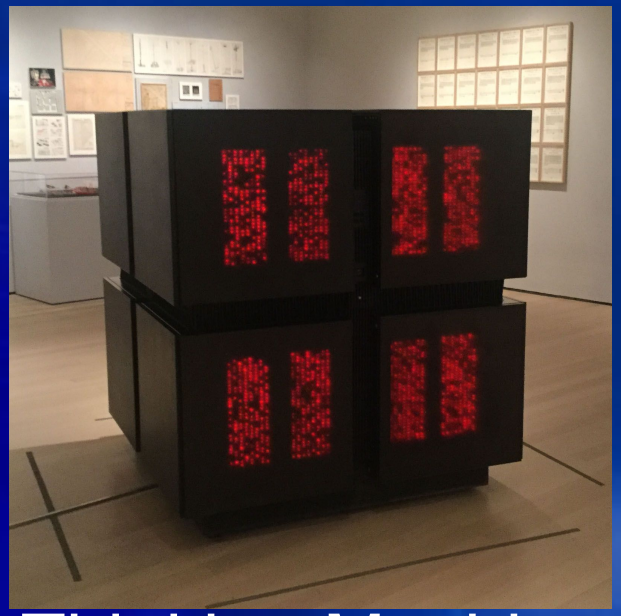


A supercomputer?

1980-2000: Supercomputers

Exotic hardware

Silicon Graphics
1982-2009



Thinking Machine
1983-1994

NCUBE
1983-2005

1980-2000: Supercomputers

Exotic hardware

Silic
1

Use commodity off-the-shelf hardware.

- Advantages:
  - Inexpensive to purchase
  - Low maintenance costs
- Disadvantages:
  - Fails often

NCUBE
1983-2005

Year 2025: Tianhe-2/3

They do not give up!

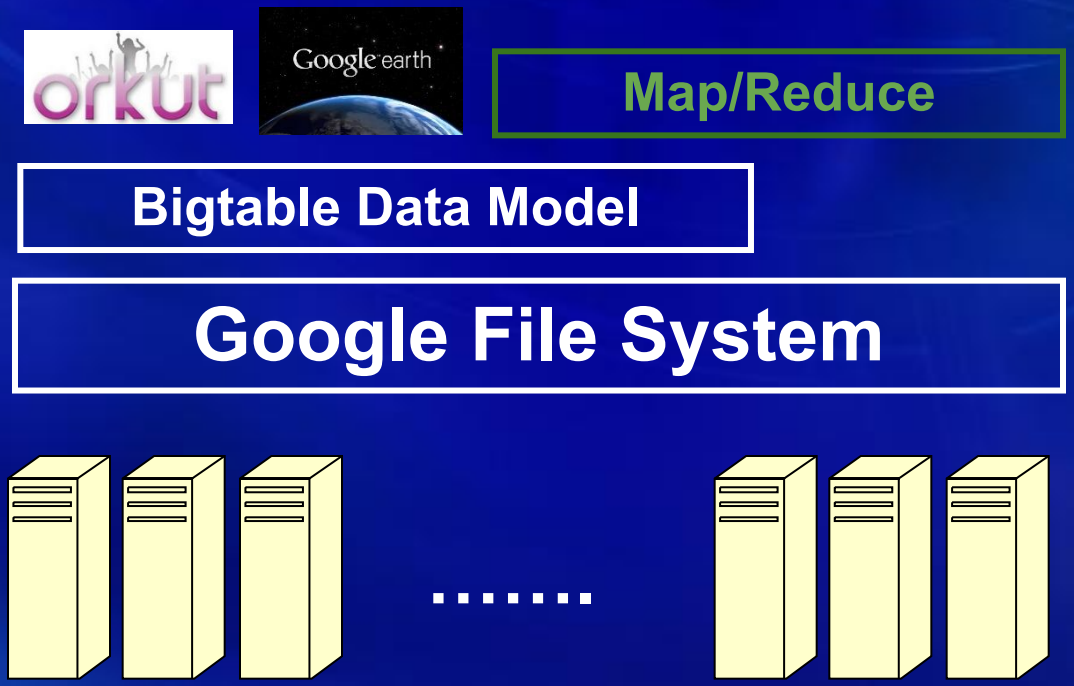
Year 2025: Tianhe-2/3

Unfortunately, the exotic hardware guys do NOT give up so easily.

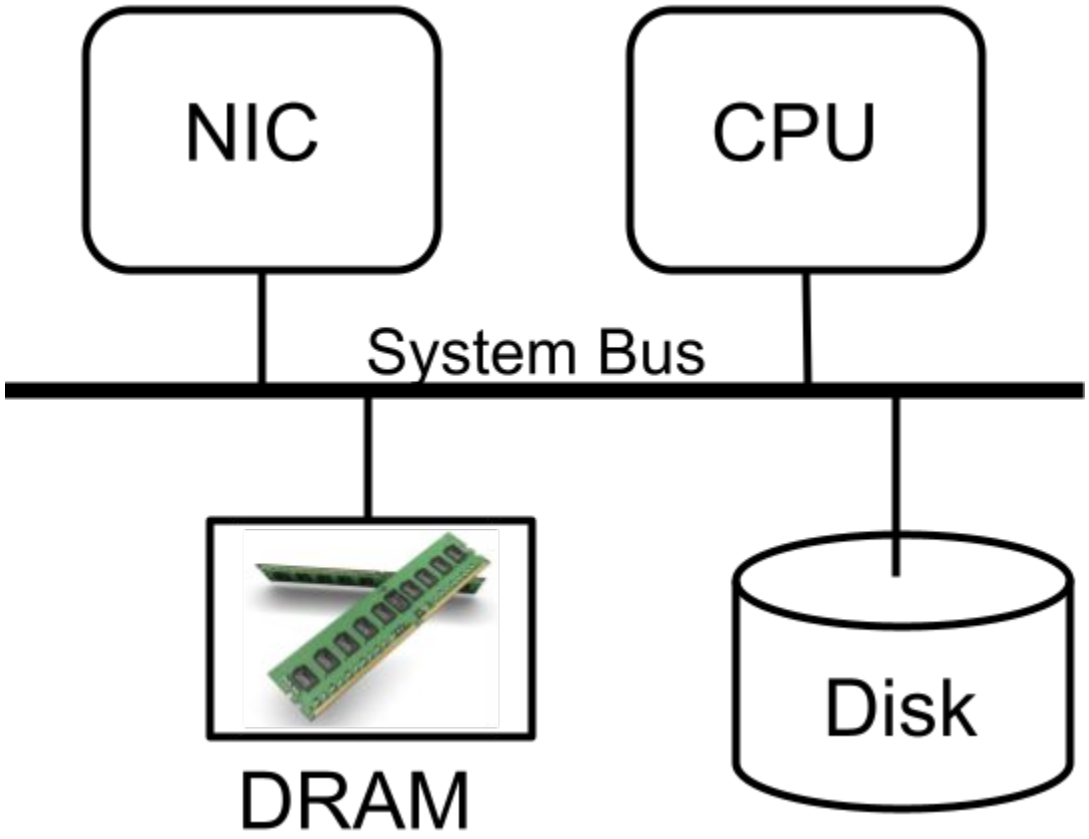
Year 2025 and we continue to see papers arguing that Google and other search engines should use exotic supercomputers.

Big Picture

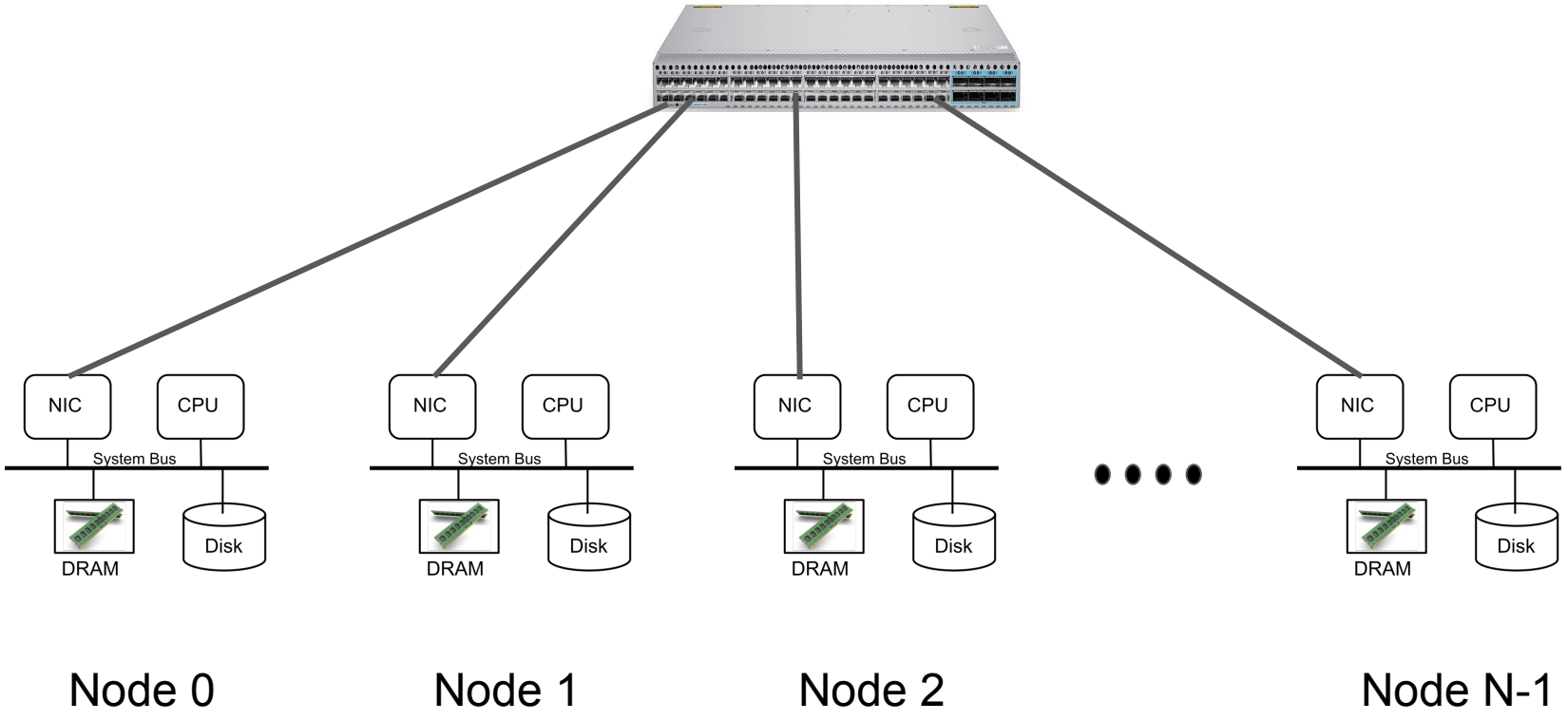
- A shared-nothing architecture consisting of thousands of nodes!
  - A node is an off-the-shelf, commodity PC.



A Node

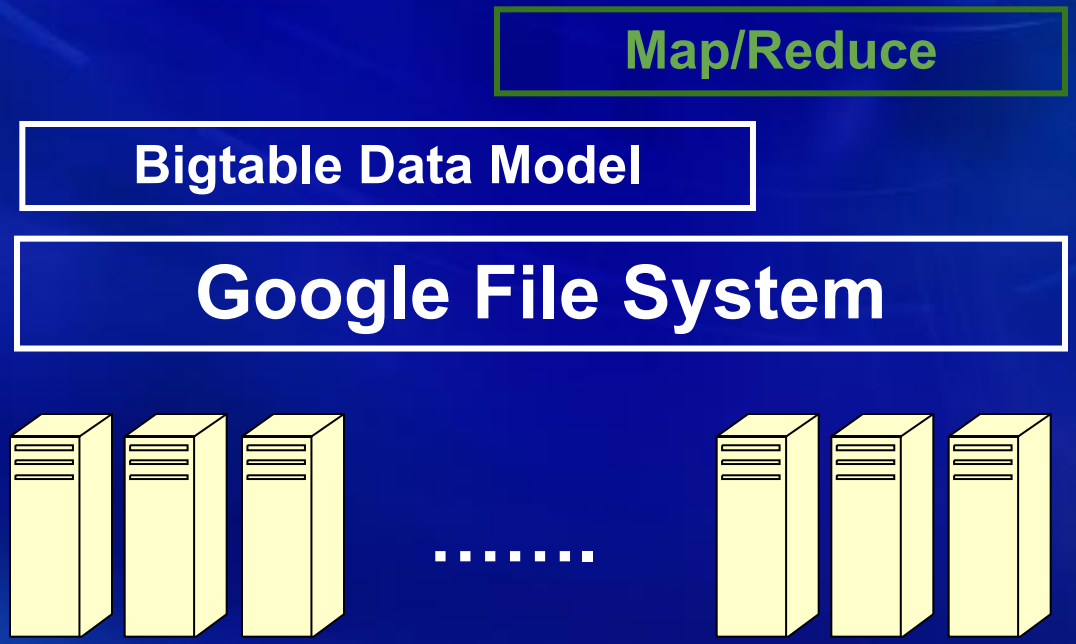


A Shared-Nothing Architecture



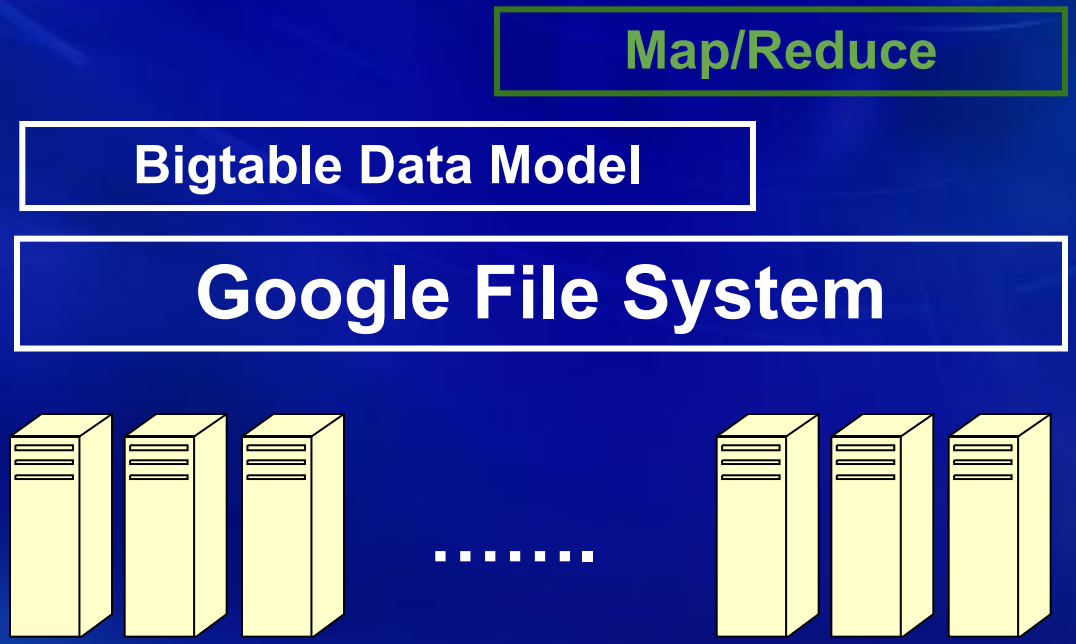
How is Map/Reduce Used? Examples include

- Inverted Index
- Different representations of the graph structure of web documents: BFS
- Summaries of the number of pages crawled per host
- Set of most frequent queries in a given day



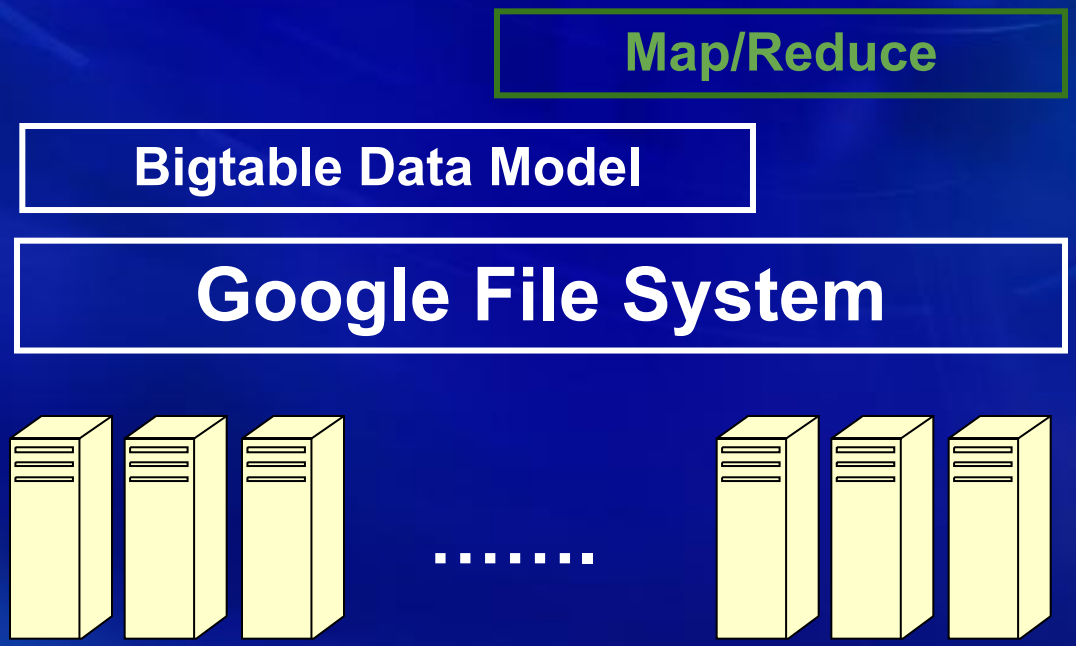
What is the challenge?

- Yes, workload is simple
- However, input data is large and computation takes a very long time on one node, several human life times.



Map/Reduce Solution

- Parallelism
- Distribute work across 100s/1000s machines
- Even with parallelism, a job may require days and weeks
- Servers may restart/fail: What would Gamma do?

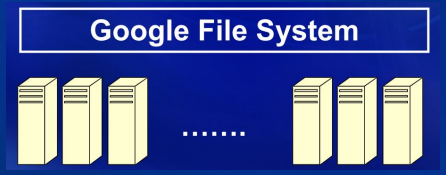


References

- Map Reduce
  - Dean and Ghemawat. MapReduce: Simplified Data Processing on Large Clusters. Communications of the ACM, Vol. 51, No. 1, January 2008.
- Bigtable
  - Chang et. al. Bigtable: A Distributed Storage System for Structured Data. In OSDI 2006.
- GFS
  - Ghemawat et. al. The Google File System. In SOSP 2003.

Questions?

GFS: Interfaces

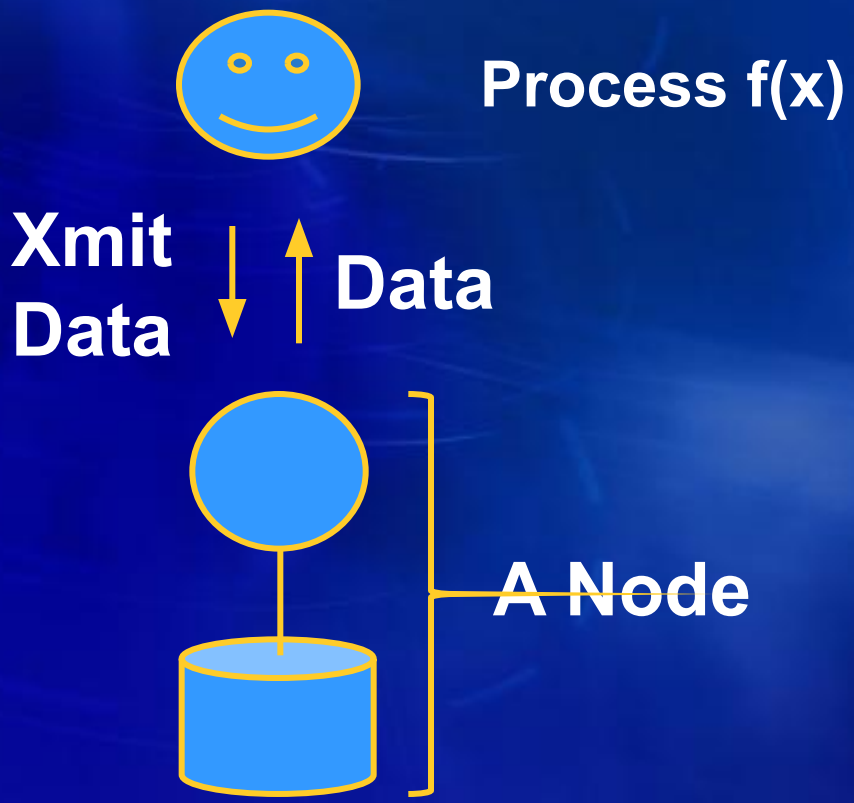


- Create, delete, open, close, read, and write files.
- Snapshot a file:
  - Create a copy of the file.
- Record append operation:
  - Allows multiple clients to append data to the same file concurrently, while guaranteeing the atomicity of each individual client's append.

- Key features:
  - A file has a fixed degree of replication d.
  - Partitions a file into fixed sized chunks.
  - Each chunk of a file is replicated across d nodes.

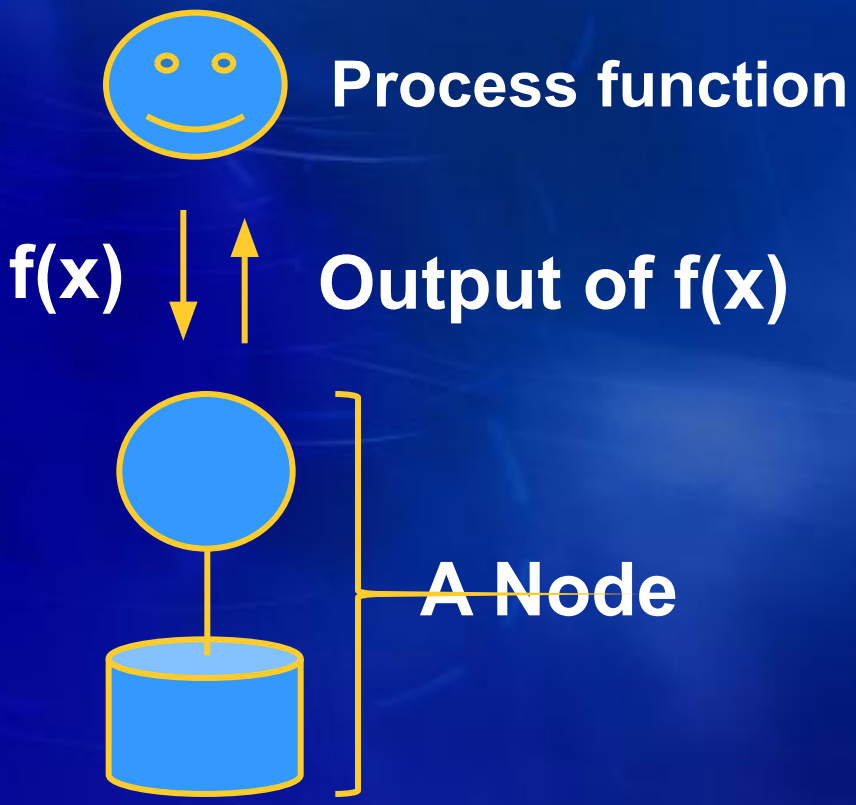
Data Shipping

- Client retrieves data from the node.
- Client performs computation locally.
- Limitation: Dumb servers, utilizes the limited network bandwidth.

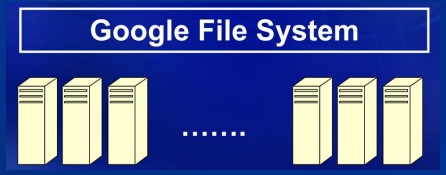


Function Shipping

- Client ships the function to the node for processing.
- Relevant data is sent to client.
- Function f(x) should produce less data than the original data stored in the database.
- Minimizes demand for the network bandwidth.

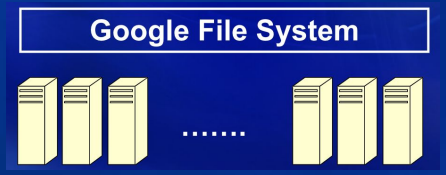


GFS Quiz



- True or False: Given thousands of crawlers retrieving web pages of thousands companies, GFS stores each retrieved web page in a file, generating billions of approximately KB-sized files

GFS Quiz



- False.
  - Retrieved web pages are stored in a file, generating several multi-GB files.
- What is the most frequent operation on a file?
  - Append to the file
  - Overwrite a portion of the file
  - Delete a portion of the file

Overview: Map/Reduce (Hadoop)

- A programming model to make parallelism transparent to a programmer.
  - Programmer specifies:
    - a map function that processes a key/value pair to generate a set of intermediate key/value pairs.
      - Divides the problem into smaller "intermediate key/value" sub-problems.
    - a reduce function to merge all intermediate values associated with the same intermediate key.
      - Solve each sub-problem.
    - Final results might be stored across R files.
  - Run-time system takes care of:
    - Partitioning the input data across nodes,
    - Scheduling the program's execution,
    - Node failures,
    - Coordination among multiple nodes.

Overview: Map/Reduce (Hadoop)

- A programming model to make parallelism transparent to a programmer.
  - Programmer specifies:
    - a map function that processes a key/value pair to generate a set of intermediate key/value pairs.
      - Divides the problem into smaller "intermediate key/value" sub-problems.
    - a reduce function to merge all intermediate values associated with the same intermediate key.
      - Solve each sub-problem.
    - Final results might be stored across R files.
  - Run-time system hides parallelism:
    - Partitioning the input data across nodes,
    - Scheduling the program's execution,
    - Node failures,
    - Coordination among multiple nodes.

Hides Parallelism

Example

- Counting word occurrences:
  - Input document is NameList and its content is: "Jim Shahram Betty Jim Shahram Jim Shahram"
  - Desired output:
    - Jim: 3
    - Shahram: 3
    - Betty: 1
- How?

```
Map(String doc_name, String doc_content)
    // doc_name is document name, NameList
    //doc_content is document content, "Jim Shahram …"
    For each word w in value
        EmitIntermediate(w, "1");

    Map (NameList, "Jim Shahram Betty …") emits:
        [Jim, 1], [Shahram, 1], [Betty, 1]
    A hash function may split different tokens across M different "Worker" processes.

Reduce (String key, Iterator values)
    // key is a word
    // values is a list of counts
    Int result = 0;
    For each v in values
        result += ParseInt(v);
    Emit(AsString(result));

    Reduce ( "Jim", {"1", "1", "1"} ) emits "Jim: 3"
```

Other Examples

- Distributed Grep:
  - Map function emits a line if it matches a supplied pattern.
  - Reduce function is an identity function that copies the supplied intermediate data to the output.
- Count of URL accesses:
  - Map function processes logs of web page requests and outputs <URL, 1>,
  - Reduce function adds together all values for the same URL, emitting <URL, total count> pairs.
- Reverse Web-Link graph; e.g., all URLs with reference to http://dblab.usc.edu:
  - Map function outputs <tgt, src> for each link to a tgt in a page named src,
  - Reduce concatenates the list of all src URLS associated with a given tgt URL and emits the pair: <tgt, list(src)>.
- Inverted Index; e.g., all URLs with “585” as a word:
  - Map function parses each document, emitting a sequence of <word, doc_ID>,
  - Reduce accepts all pairs for a given word, sorts the corresponding doc_IDs and emits a <word, list(doc_ID)> pair.
  - Set of all output pairs forms a simple inverted index.

Questions?

MapReduce

- Input: F = {f1, f2, …, fn}, user provided functions M and R
  - M(fi) → {[K1, V1], [K2, V2], … }
    - [Jim, 1], [Shahram, 1], [Betty, 1], …
    - [Jim, {“1”, “1”, “1”}], [Shahram, {“1”, “1”, “1”}], [Betty, {“1”}]
  - R(Ki, ValueSet) → [Ki, R(ValueSet)]
    - [Jim, “3”], [Shahram, “3”], [Betty, “1”]

```
map(String key, String value):
    // key: document name
    // value: document contents
    for each word w in value:
        EmitIntermediate(w, “1”);

reduce(String key, Iterator values):
    // key: a word
    // values: a list of counts
    int result = 0;
    for each v in values:
        result += ParseInt(v);
    Emit(AsString(result));
```

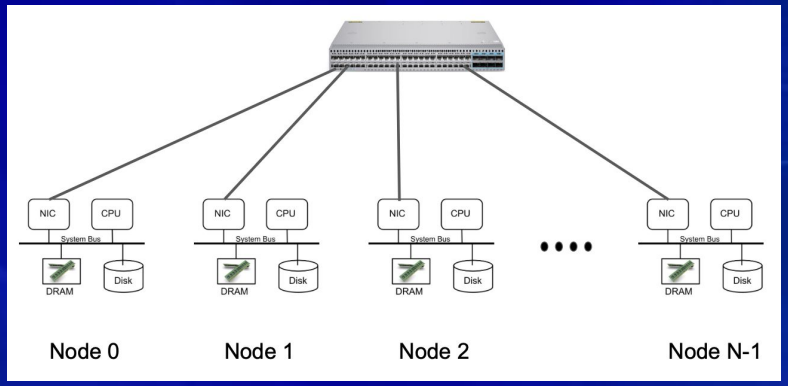
In addition, the user writes code to fill in a *mapreduce specification* object with the names of the input and output files and optional tuning parameters. The user then invokes the *MapReduce* function, passing it to the specification object. The user's code is linked together with the MapReduce library (implemented in C++). Our original MapReduce paper contains the full program text for this example [8].

Map/Reduce: Objective

- Hide details of parallelizing the computation:
  - Distribute the data
  - Balance load
  - Make forward progress in the presence of failures
- Support arbitrarily complex task.

Implementation

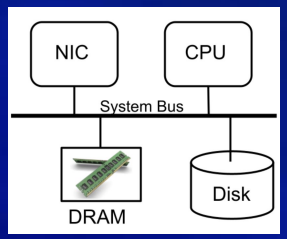
- Target environment:
  - Commodity PCs connected using a switched Ethernet.
    - Storage for the “Map” function is provided by inexpensive IDE disks attached to each PC.



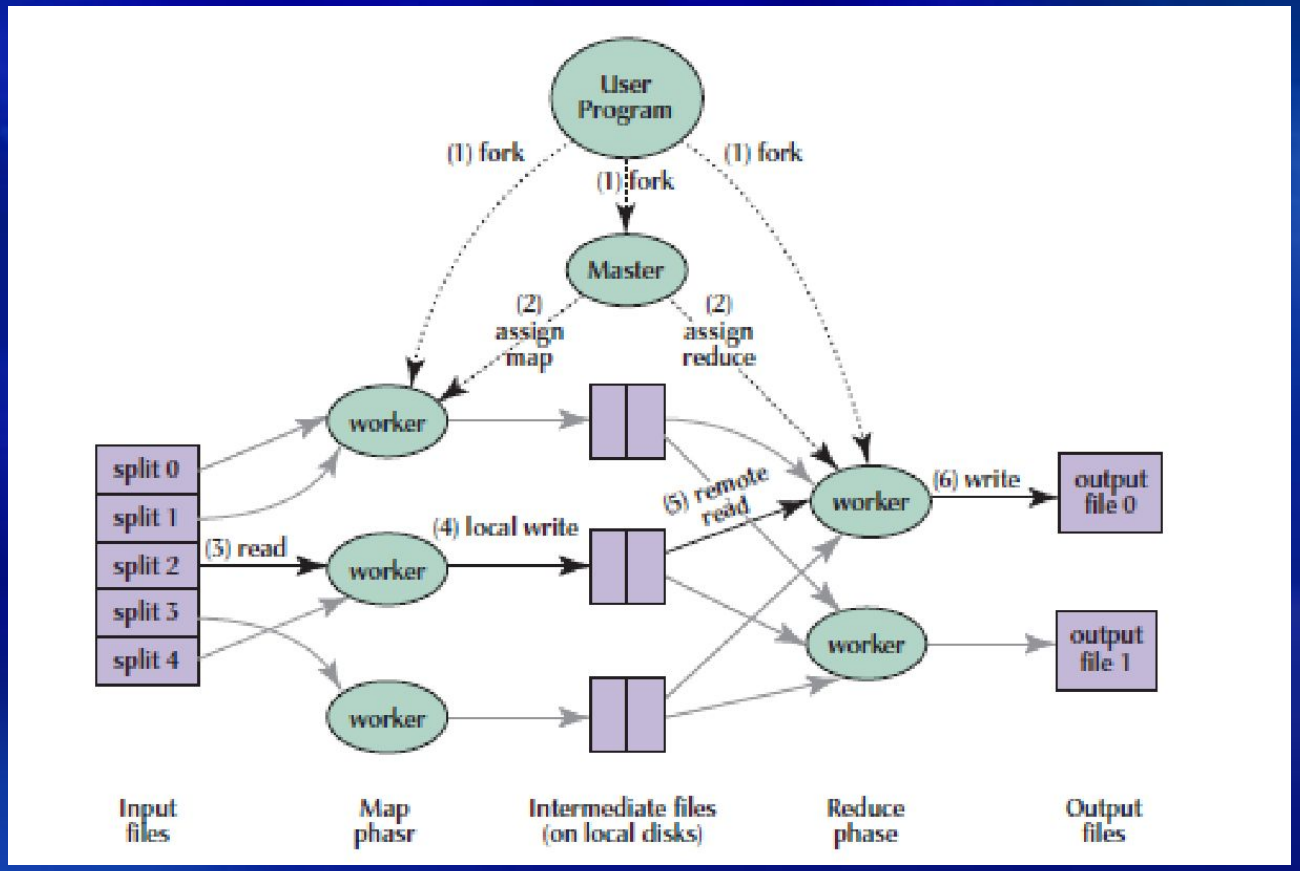
  - GFS manages data stored across PCs.
  - A scheduling system accepts jobs submitted by users, each job consists of a set of tasks, and the scheduler maps tasks to a set of available machines within a cluster.

Execution

- Map invocations are distributed across multiple machines by automatically partitioning the input data into a set of M splits.
- Reduce invocations are distributed by partitioning the intermediate key space into R pieces using a hash function: hash(key) mod R.
  - R and the partitioning function are specified by the programmer.



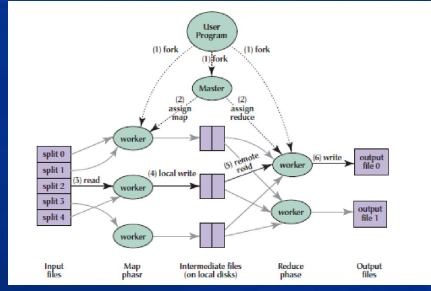
A worker server



Output of Execution

- Each map task produces R output files, one per reduce task, with file name specified by the programmer.
  - These files are written to local disk - no replication. Node fails then this data becomes unavailable.
- Programmers are not required to combine the final output files of the reduce task into one file – they may pass these as input to another MapReduce call (or use them with another distributed application that is able to deal with input that is partitioned into multiple files).

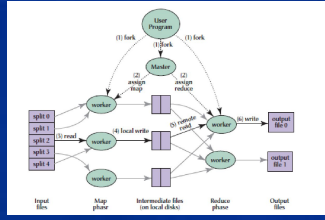
Execution



- Important details:
  - Output of Map task is stored on the local disk of the PC (Worker) executing the task.
    - A Map task produces R such files on its local disk.
  - Output of Reduce task is stored in the GFS. High availability via replication.
    - The filename of the output produced by a reduce task is deterministic.
    - When a reduce task completes, the reduce worker atomically renames its temporary output file to the final output file.
    - NOTE: If the same reduce task executes on multiple machines, multiple renames calls will be executed for the same output file.

Questions?

Master



- Propagates location of intermediate file regions from map tasks to reduce tasks. For each completed map task, master stores the location and sizes of the R produced intermediate files.
- Pushes location of the R produced intermediate files to the workers with in-progress reduce tasks.
- For each map task and reduce task, master stores the possible states: idle, in-progress, or completed.
- Master takes the location of input files (GFS) and their replicas into account. It strives to schedule a map task on a machine that contains a replica of the corresponding input file (or near it).
  - Minimize contention for the network bandwidth.
- Termination condition: All map and reduce tasks are in the “completed” state.

Worker Failures

- Failure detection mechanism: Master pings workers periodically.
- An in-progress Map or Reduce task on a failed worked is reset to idle and eligible for rescheduling.
- Completed Map task on a failed worker must also be re-executed because its output are stored on the local disk.
- Completed Reduce tasks do not need to be re-executed because their output is stored in GFS, a global file system.

> When a map task is executed first by worker *A* and then later executed by worker *B* (because *A* failed), all workers executing reduce tasks are notified of the reexecution. Any reduce task that has not already read the data from worker *A* will read the data from worker *B*.

Master Failure

- Abort the MapReduce computation.
- Client may check for this condition and retry the MapReduce operation.

- Alternative: the master checkpoint its data structures, enabling a new instance to resume from the last checkpoint state.

Sequential versus Parallel Execution

- Is the results of a sequential execution the same as the parallel execution with failures?

Sequential versus Parallel Execution

- Is the results of a sequential execution the same as the parallel execution with failures?
  - Depends on the application.
- If Map and Reduce operators are deterministic functions of their input values THEN the output is the same as a non-faulting execution. How?
  - When a map task completes, the worker sends a message to the master and includes the name of the R temporary files in the message.
  - If the master receives a completion message for an already completed map task, it ignores the message. Otherwise, it records the names of R files in a master data structure (for use by the reduce tasks).
  - Output of Reduce task is stored in the GFS. High availability via replication.
    - The filename of the output produced by a reduce task is deterministic.
    - When a reduce task completes, the reduce worker atomically renames its temporary output file to the final output file.
    - If the same reduce task executes on multiple machines, multiple renames calls will be executed for the same output file.

Sequential versus Parallel Execution

- **What if Map and Reduce operators are NOT deterministic functions of their input values?**

havior. When the *map* and/or *reduce* operators are non-deterministic, we provide weaker but still reasonable semantics. In the presence of non-deterministic operators, the output of a particular reduce task R_1 is equivalent to the output for R_1 produced by a sequential execution of the non-deterministic program. However, the output for a different reduce task R_2 may correspond to the output for R_2 produced by a different sequential execution of the non-deterministic program.

Consider map task M and reduce tasks R_1 and R_2. Let $e(R_i)$ be the execution of R_i that committed (there is exactly one such execution). The weaker semantics arise because $e(R_1)$ may have read the output produced by one execution of M and $e(R_2)$ may have read the output produced by a different execution of M.

Questions?

Load Balancing

- Similar to HRPS, values of M and R are much larger than the number of worker machines.
  - When a worker fails, the many tasks assigned to it can be spread out across all the other workers.
- Master makes O(M+R) scheduling decisions and maintains O(MR) states in memory.
- Practical guidelines:
  - M is chosen so that each task is roughly 16-64 MB of input data,
  - R is a small multiple of the number of worker machines (R=5000 with 2000 worker machines).

Load Imbalance: Stragglers

- Stragglers are those tasks that take an unusually long time to complete one of the last few map or reduce tasks.
  - Load imbalance is a possible reason.
- Master schedules backup executions of the remaining in-progress tasks when a MapReduce operation is close to completion.
- Task is marked as completed whenever either the primary or the backup execution completes.
- Significant improvement in execution time; 44% with sort.

Function Shipping

- Programmer may specify a "Combiner" function that does partial merging of data produced by a Map task.
- The output of a combiner function is written to an intermediate file that is consumed by the reduce task.
- Typically, the same code is used to implement both the combiner and the reduce functions.
  - Example: With the word count example, there will be many instances with ["Jim", 1] because "Jim" is more common than "Shahram". Programmer writes a "Combiner" function to enable a Map task to produce ["Jim", 55]. In order for this to work, the reduce function should be commutative and associative.

> Partial combining significantly speeds up certain classes of MapReduce operations. Appendix A contains

How to Debug?

- How does a programmer debug his or her MapReduce application?

How to Debug?

- How does a programmer debug his or her MapReduce application?
  - Alternative implementation of the MapReduce library that sequentially executes all of the work on the local machine.
  - Programmer may focus on a particular map tasks.
- What if the input data is causing failures?

How to Debug?

- How does a programmer debug his or her MapReduce application?
  - Alternative implementation of the MapReduce library that sequentially executes all of the work on the local machine.
  - Programmer may focus on a particular map tasks.
- What if the input data is causing failures?
  - Optional mode of execution where the MapReduce library detects which records cause deterministic crashes and skips these record.
  - Master knows about these records and the programmer may retrieve them for further analysis.

Monitoring of MapReduce

- **Very important to have eyes that can see:**

The master runs an internal HTTP server and exports a set of status pages for human consumption. The status pages show the progress of the computation, such as how many tasks have been completed, how many are in progress, bytes of input, bytes of intermediate data, bytes of output, processing rates, etc. The pages also contain links to the standard error and standard output files generated by each task. The user can use this data to predict how long the computation will take, and whether or not more resources should be added to the computation. These pages can also be used to figure out when the computation is much slower than expected.

In addition, the top-level status page shows which workers have failed, and which map and reduce tasks they were processing when they failed. This information is useful when attempting to diagnose bugs in the user code.

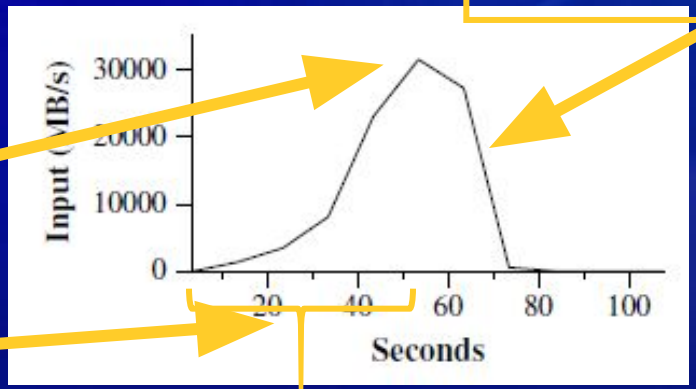
Questions?

Performance Numbers

- A cluster consisting of 1800 PCs:
  - 2 GHz Intel Xeon processors
  - 4 GB of memory
    - 1-1.5 GB reserved for other tasks sharing the nodes.
  - 320 GB storage: two 160 GB IDE disks
- Grep 1 TB (10 billion records) of data looking for a rare three-character pattern:
  - M=15000 64 MB, R=1, record size = 100 bytes, results set size is 92,337
  - Execution time is 150 Seconds.

Map tasks start to finish.

1764 workers are assigned!



Time to schedule tasks; startup.

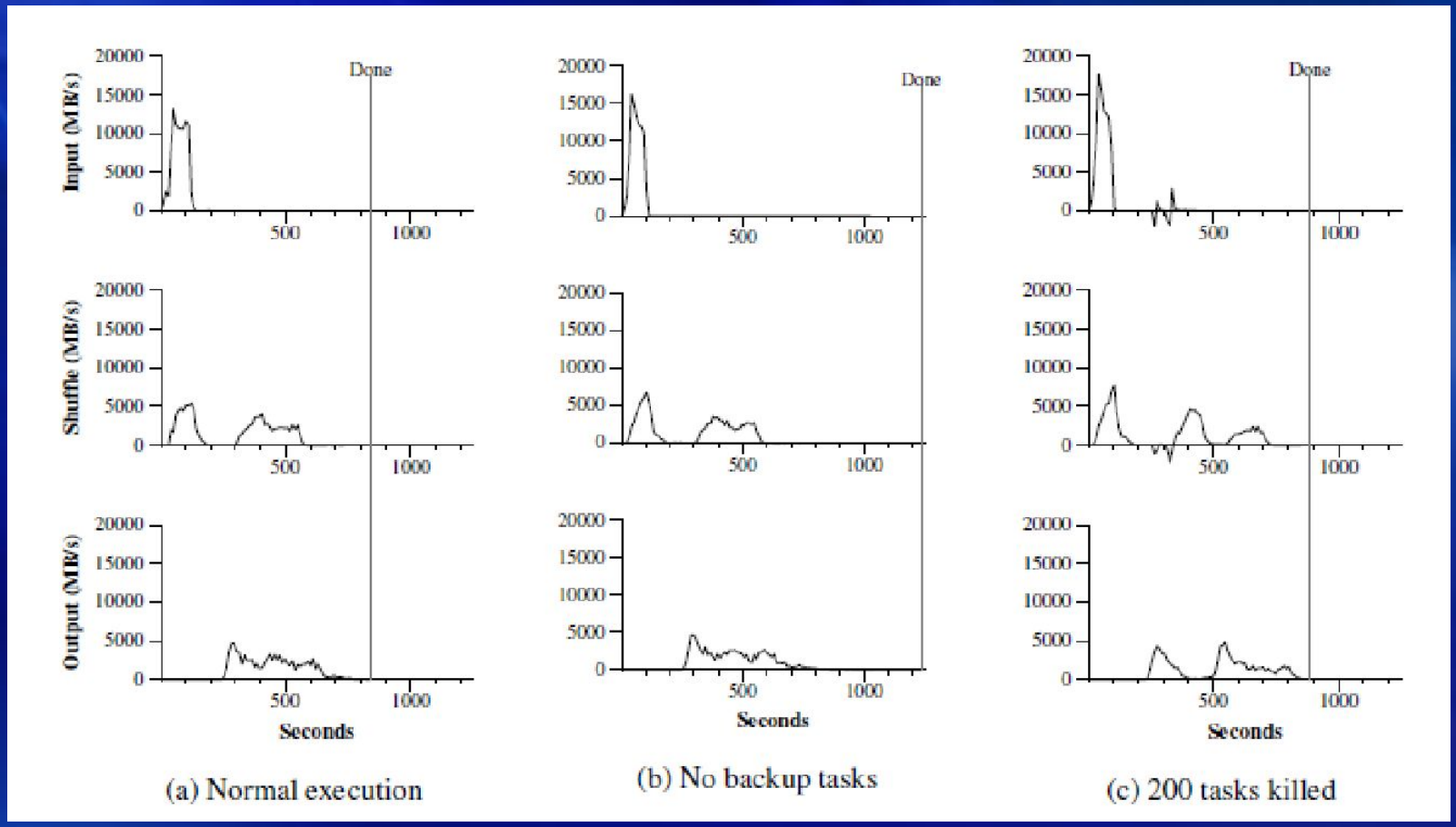
Startup with Grep

- Startup includes:
  - Propagation of the program to all worker machines,
  - Delays interacting with GFS to open the set of 1000 input files,
  - Information needed for the locality optimization.

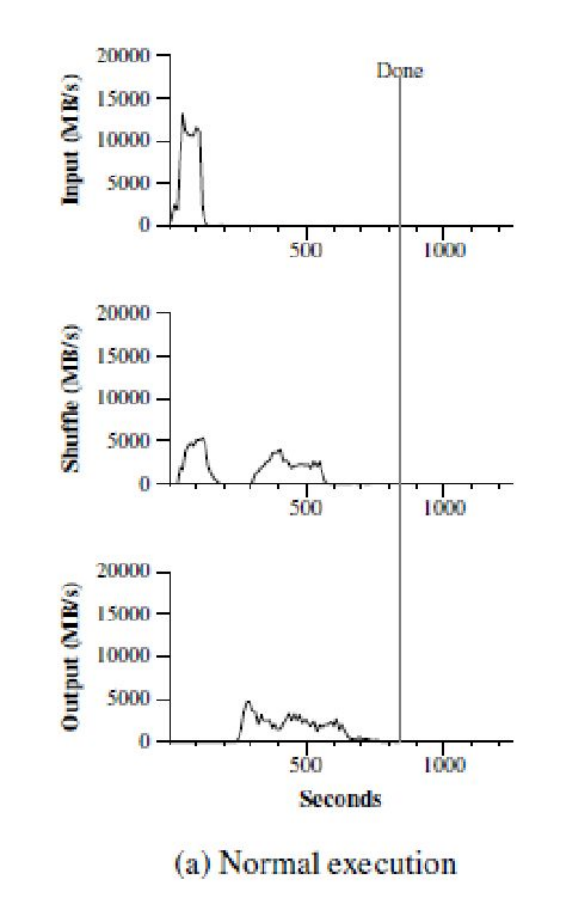
Sort

- Map function extracts a 10-byte sorting key from a text line, emitting the key and the original text line as the intermediate key/value pair.
  - Each intermediate key/value pair will be sorted.
- Identity function as the reduce operator.
  - R = 4000.
  - Partitioning information has built-in knowledge of the distribution of keys.
    - If this information is missing, add a pre-pass MapReduce to collect a sample of the keys and compute the partitioning information.
- Final sorted output is written to a set of 2-way replicated GFS files.

Sort Results



Sort Results: Definitions



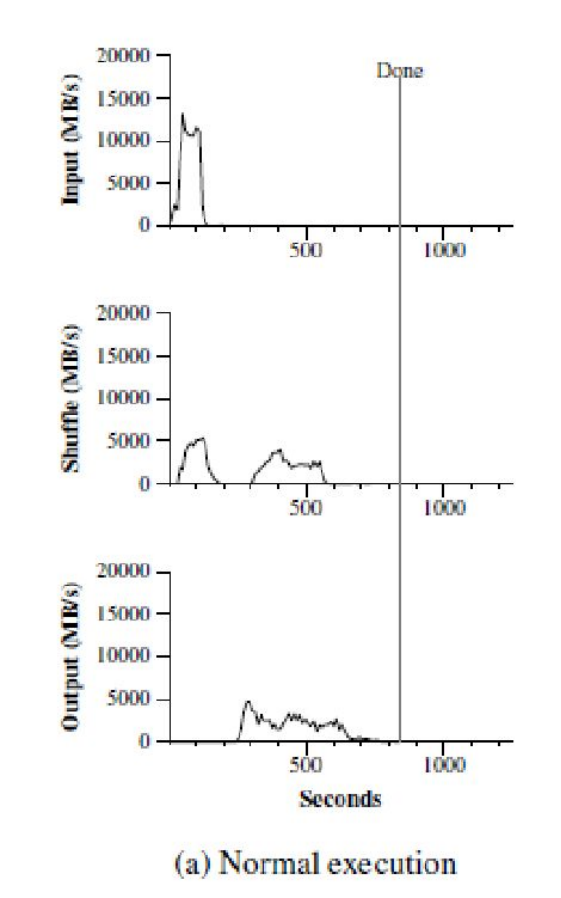
(a) Normal execution

Rate at which map workers read data from the local disks.

Rate at which data is sent over the network.

Rate at which reduce workers write the final output to the GFS, degree of replication = 2.

Sort Results: Interesting Observations

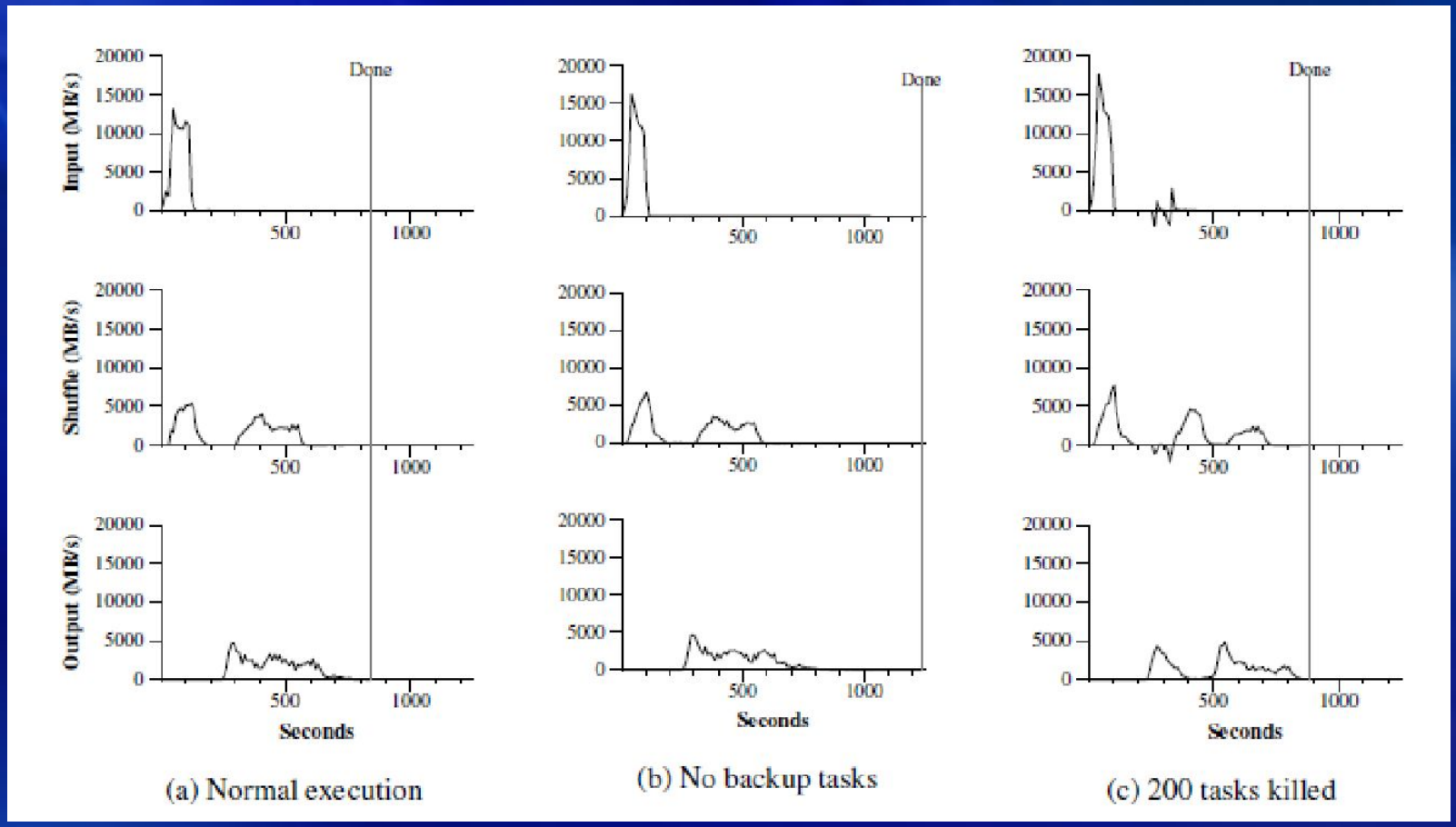


(a) Normal execution

A few things to note: the input rate is higher than the shuffle rate and the output rate because of our locality optimization – most data is read from a local disk and bypasses our relatively bandwidth constrained network. The shuffle rate is higher than the output rate because the output phase writes two copies of the sorted data (we make two replicas of the output for reliability and availability reasons). We write two replicas because that is the mechanism for reliability and availability provided by our underlying file system. Network bandwidth requirements for writing data would be reduced if the underlying file system used erasure coding [14] rather than replication.

Sort Results

- Backup tasks reduce execution time by 44%.
- Servers function properly once tasks are killed.



MapReduce: Conclusions

	Aug. '04	Mar. '06	Sep. '07
Number of jobs (1000s)	29	171	2,217
Avg. completion time (secs)	634	874	395
Machine years used	217	2,002	11,081
map input data (TB)	3,288	52,254	403,152
map output data (TB)	758	6,743	34,774
reduce output data (TB)	193	2,970	14,018
Avg. machines per job	157	268	394
Unique implementations			
map	395	1958	4083
reduce	269	1208	2418

ber 2004, to about 4000 in March 2006. MapReduce has been so successful because it makes it possible to write a simple program and run it efficiently on a thousand machines in a half hour, greatly speeding up the development and prototyping cycle. Furthermore, it allows programmers who have no experience with distributed and/or parallel systems to exploit large amounts of resources easily.

MapReduce

- Any questions?

Discussion

Question 1:

- Master takes the location of input files (GFS) and their replicas into account. It strives to schedule a map task on a machine that contains a replica of the corresponding input file (or near it).
  - Minimize contention for the network bandwidth.
- Can the reduce tasks be scheduled on the same nodes that are holding the intermediate data on their local disks to further reduce network traffic?

Answer to Question 1

- Probably not because every Map task will have some data for each Reduce task.
- A Map task produces R output files, each to be consumed by one reduce tasks.
  - If there is 1 Map task and 10 Reduce tasks then the 1 Map task produces 10 output files.
    - Each file resembles partitioning of an intermediate key/value pair, e.g., intermediate key % R.
  - If there is 200 Map tasks and 10 Reduce tasks, the Map phase produces 2000 files (10 files produced by each Map task).
    - Each reduce task processes the 200 files (produced by 200 different Map tasks) that map to the same partitioning, e.g., intermediate key % R.
    - The master may assign a reduce task to one node; it must pick one of the 200 Map tasks.

Question 1.a

- Probably not because every Map task will have some data for each Reduce task.
- A Map task produces R output files, each to be consumed by one reduce tasks.
  - If there is 1 Map task and 10 Reduce tasks then the 1 Map task produces 10 output files.
    - Each file resembles partitioning of an intermediate key/value pair, e.g., intermediate key % R.
  - If there is 200 Map tasks and 10 Reduce tasks, the Map phase produces 2000 files (10 files produced by each Map task).
    - Each reduce task processes the 200 files (produced by 200 different Map tasks) that map to the same partitioning, e.g., intermediate key % R.
    - The master may assign a reduce task to one node; it must pick one of the 200 Map tasks.
    - What if there are 200 Map tasks and 200 Reduce tasks?

Answer to Question 1.a

- Probably not because every Map task will have some data for each Reduce task.
- A Map task produces R output files, each to be consumed by one reduce tasks.
  - If there is 1 Map task and 10 Reduce tasks then the 1 Map task produces 10 output files.
    - Each file resembles partitioning of an intermediate key/value pair, e.g., intermediate key % R.
  - If there is 200 Map tasks and 10 Reduce tasks, the Map phase produces 2000 files (10 files produced by each Map task).
    - Each reduce task processes the 200 files (produced by 200 different Map tasks) that map to the same partitioning, e.g., intermediate key % R.
    - The master may assign a reduce task to one node; it must pick one of the 200 Map tasks.
    - What if there are 200 Map tasks and 200 Reduce tasks?
      - There will be a total of 40,000 files to process.
      - Each reduce task must retrieve 200 different files from 200 different Map tasks.
      - Scheduling a reduce task on one node requires transmission of 199 other files across the network.

Question 2

- Given R reduce tasks, once reduce task ri is assigned to a worker, all partitioned intermediate key values that map to ri MUST be sent to this worker. Why?

Question 2

- Given R Reduce tasks, once Reduce task ri is assigned to a worker, all partitioned intermediate key values logically assigned to ri MUST be sent to this worker. Why?
  - Reduce task ri does aggregation and must have all instances of the intermediate keys produced by different Map tasks.
  - In our example, [“Jim”, “1 1 1”] produced by five different map tasks must be directed to the same reduce task so that it computes [“Jim”, “15”] as its output.
  - If directed to five different reduce tasks, each reduce task will produce [“Jim”, “3”] and there is no mechanism to merge them together!

Question 3

- Are the renaming operations at the end of a Reduce task protected by locks? Is it possible for a file to become corrupted if two threads attempt to rename it to the same name at essentially the same time? Or does the rename operation happen so fast that the chances of this happening are very remote?

Question 3

- Are the renaming operations at the end of a Reduce task protected by locks? Is it possible for a file to become corrupted if two threads attempt to rename it to the same name at essentially the same time? Or does the rename operation happen so fast that the chances of this happening are very remote?
  - The rename operations are performed on two different files, produced by different Reduce tasks that performed the same computation.
    - A file produced by the Reduce task corresponds to a range, i.e., a tablet of Bigtable.
  - To update the meta-data, the tablet server must update the meta-data on Chubby.
    - There is one instance of this meta-data.
    - Chubby serializes the rename operations.

Question 4

- I have a hard time picturing a useful non-deterministic function. Can you give an example of a non-deterministic function that could be implemented by Map/Reduce.
  - How to construct a non-deterministic function?
  - What are some of the examples that may use such a non-deterministic function?

Question 4

- I have a hard time picturing a useful non-deterministic function. Can you give an example of a non-deterministic function that could be implemented by Map/Reduce.
  - How to construct a non-deterministic function?
    - A computation that uses a random number generator.
      - An optimization with a large search space such that it requires heuristic search starting with a randomly chosen node in the space.
  - What are some of the examples that may use such a non-deterministic function?
    - Given a term not-encountered before, what are the best advertisements to offer the user to maximize profits?